Introduction to Deep Learning

Magda Gregorová

magda.gregorova@thws.de

March 2026

Contents

Part I. Foundations

1. What is Deep Learning?

1.1. Demonstration

▹ Session 1, Slides 1-12

Before any theory, consider the following experiment. A large language model - specifically Claude - is prompted with a single sentence:

> *"Write me a PyTorch script that trains a neural network to classify CIFAR-10 images into 10 classes. Use a simple MLP architecture. Print the training loss each epoch and plot a few example predictions with their true and predicted labels at the end."*

The model produces a complete, runnable Python script. When executed, the script downloads the CIFAR-10 dataset - 60,000 colour images of 32×32 pixels across 10 classes - trains a neural network, and achieves reasonable classification accuracy. A second prompt requesting a CNN instead of an MLP produces a different script that achieves noticeably higher accuracy on the same task.

This demonstration works. It is genuinely easy. And therein lies the problem: *what just happened?*

The script contains weights, layers, a loss function, a training loop, gradient updates. None of these were explained. The goal of this course is to open that black box - not to prompt better, but to think deeper.

1.2. What is Deep Learning?

▹ Session 1: What is Deep Learning?

Artificial intelligence, machine learning, and deep learning are often used interchangeably in popular discourse, but they refer to distinct and nested concepts.

Artificial intelligence is the broadest term - it encompasses any technique that enables machines to exhibit behaviour we would consider intelligent, including rule-based expert systems, search algorithms, and learned models.

Machine learning is a subfield of AI in which systems learn from data rather than being programmed with explicit rules. Instead of writing down every decision, we provide examples and let the system discover the underlying patterns.

Deep learning is a subfield of machine learning that uses neural networks with many layers - so-called deep neural networks. The depth allows the network to learn increasingly abstract representations of the input data.

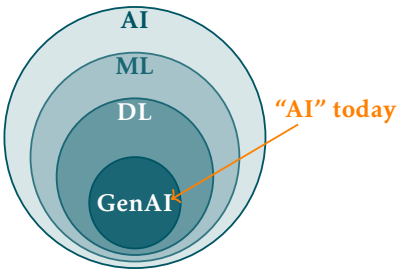

Figure 1.1: The relationship between AI, machine learning, deep learning, and generative AI.

Generative AI - encompassing large language models as well as image, video, and music generation systems - is a specific and currently prominent subset of deep learning. What the general public refers to as "AI" today is almost exclusively generative AI, which sits at the innermost level of this hierarchy.

Definition 1 (Deep Learning)**.** Deep learning is a class of machine learning methods that use multi-layered neural networks to learn hierarchical representations directly from raw data.

This definition will sharpen as the course progresses. For now it is sufficient to hold it loosely - by the end of the course every word in it will have a concrete meaning.

1.3. Brief History

▹ Session 1: A Brief History of Deep Learning

Figure 1.2 shows the key milestones in the development of deep learning alongside the periods of reduced funding and interest known as AI winters. For a comprehensive overview of the field's history see LeCun, Bengio, and G. Hinton 2015 and Goodfellow, Bengio, and Courville 2016.

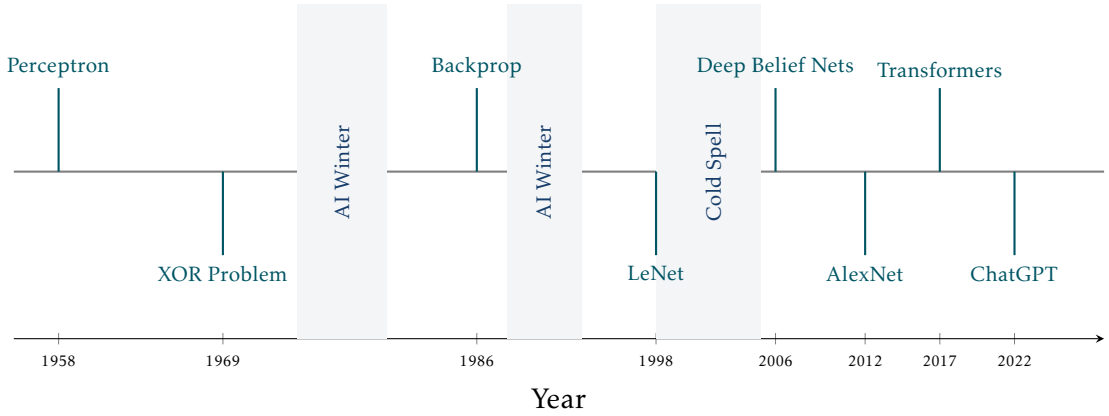

Figure 1.2: Key milestones in the history of deep learning. Shaded regions indicate AI winters and the cold spell of the early 2000s.

1958 - Perceptron. Frank Rosenblatt 1958 introduced the perceptron - the first trainable neural network. It could learn to classify linearly separable patterns by adjusting weighted connections between inputs and a single output unit. The result generated enormous excitement and equally enormous overpromising about the future of machine intelligence. Rosenblatt himself claimed the perceptron would eventually be able to walk, talk, see, write, reproduce itself, and be conscious of its existence - a level of optimism that would later contribute to the severity of the backlash.

The years following the perceptron saw genuine progress. The ADALINE model Widrow and Hoff 1960 introduced the least mean squares learning rule, which remains in use today. The concept of multilayer networks was understood theoretically, and researchers were actively exploring how to train them. There was a widespread belief that human-level artificial intelligence was only a decade away - a belief shared by many prominent scientists and echoed in generous research funding from DARPA and other agencies.

However, the limitations of single-layer networks were becoming apparent in practice. Many of the problems researchers wanted to solve - pattern recognition, language understanding, game playing - turned out to be far harder than anticipated. The gap between the promise of the perceptron and the reality of what it could achieve was widening.

1969 - XOR Problem. Minsky and Papert 1969 published *Perceptrons*, demonstrating formally that single-layer networks could not solve problems as simple as the XOR

function. The 1970s were a difficult period for neural network research. The Lighthill Report Lighthill 1973 delivered a damning assessment of AI research in 1973, concluding that the field had failed to deliver on its promises. DARPA cut funding significantly and many researchers moved away from neural networks entirely. This was the first AI winter.

Yet work continued quietly. Paul Werbos derived the backpropagation algorithm in his 1974 PhD thesis Werbos 1974 - years before it became widely known. Fukushima 1980 introduced the Neocognitron, a hierarchical neural network inspired by the visual cortex that anticipated many ideas later formalised in convolutional networks. Hopfield 1982 networks demonstrated that recurrent networks could function as associative memories. These contributions kept the field alive through the cold years, largely unnoticed by the mainstream AI community which had shifted its focus toward symbolic reasoning and expert systems.

Japan's role in this period deserves particular mention. Fukushima's Neocognitron emerged from NHK Science and Technology Research Laboratories in Tokyo. In 1982 the Japanese government launched the ambitious Fifth Generation Computer Project Ministry of International Trade and Industry 1982, investing heavily in AI hardware and parallel computing. The project caused significant alarm in the US and UK and triggered substantial counter-investment in AI research on both sides of the Atlantic. Its ultimate failure to deliver human-level reasoning contributed to the disillusionment of the second AI winter.

1986 - Backpropagation. Rumelhart, G. E. Hinton, and Williams 1986 popularised the backpropagation algorithm, providing a practical method for training multi-layer networks by propagating error gradients backwards through the layers. The revival of backpropagation generated enormous enthusiasm. Multilayer networks were successfully applied to speech recognition, image classification, and financial forecasting. LeCun applied backpropagation to convolutional networks at Bell Labs, demonstrating practical handwriting recognition systems used by the US postal service. Hochreiter and Schmidhuber introduced the Long Short-Term Memory (LSTM) network in 1997 Hochreiter and Schmidhuber 1997, solving the vanishing gradient problem for recurrent networks and laying the foundation for sequence modelling over the following two decades.

However, the second AI winter arrived in the late 1980s and early 1990s. Expert systems - rule-based AI programs that had attracted enormous commercial investment - collapsed as their maintenance costs proved prohibitive and their brittleness became apparent. The Lisp machine market crashed in 1987. DARPA again cut funding. Neural networks hit their own practical ceiling: training deep networks remained unstable, datasets were small, and support vector machines (SVMs) - introduced by Vapnik 1995 - offered better performance on many tasks with stronger theoretical guarantees. By the mid-1990s SVMs had largely displaced neural networks as the method of choice for classification problems.

1998 - LeNet. Yann LeCun, Bottou, et al. 1998 demonstrated convolutional neural networks for handwritten digit recognition, achieving state-of-the-art results on the MNIST dataset. Despite its success, neural networks remained out of favour through the early 2000s - a period sometimes called the cold spell - as support vector machines and other kernel methods dominated the field.

This cold spell was not a complete winter - small groups continued working on deep architectures. Bengio, Delalleau, and Le Roux 2004 explored the difficulty of training

deep networks, identifying the vanishing gradient problem as the central obstacle. Hinton's group at Toronto worked on unsupervised pretraining as a way to initialise deep networks before fine-tuning with backpropagation. The theoretical groundwork for the 2006 breakthrough was being laid quietly throughout this period.

2006 - Deep Belief Nets. G. E. Hinton, Osindero, and Teh 2006 showed that deep networks could be trained effectively using a layer-wise pretraining procedure . This reignited interest in deep architectures and marked the beginning of the modern deep learning era, even before large datasets and GPU training were fully established.

The key developments of this period were infrastructural rather than algorithmic. Raina, Madhavan, and Ng 2009 demonstrated that GPUs could accelerate neural network training by an order of magnitude, making experiments that previously took weeks feasible in hours. The ImageNet dataset was released by Deng et al. 2009, providing for the first time a benchmark large and challenging enough to reveal the true potential of deep architectures. Nair and G. E. Hinton 2010 introduced the ReLU activation function, which proved dramatically more effective than sigmoid and tanh activations for training deep networks. These three developments - GPU training, large-scale data, and better activations - set the stage for AlexNet's breakthrough in 2012.

2012 - AlexNet. Krizhevsky, Sutskever, and G. E. Hinton 2012 entered a deep convolutional network trained on GPUs into the ImageNet challenge and won by a margin that shocked the field - reducing the top-5 error rate from 26% to 15% in a single year. This moment is widely regarded as the turning point that established deep learning as the dominant paradigm in machine learning.

2017 - Transformers. Vaswani et al. 2017 introduced the transformer architecture in the paper *Attention Is All You Need*, replacing recurrent computation with self-attention mechanisms. The transformer became the foundation of virtually all modern large-scale models in both language and vision.

2022 - ChatGPT. OpenAI released ChatGPT, bringing large language models to a mass audience for the first time. Within two months it had reached 100 million users, making it the fastest-growing consumer application in history and triggering a wave of public and institutional interest in artificial intelligence that continues today.

The pattern visible in Figure 1.2 is worth noting: the history of deep learning is one of recurring cycles of excitement and disappointment, punctuated by genuine breakthroughs. The current era is distinguished not just by the scale of the results but by the breadth of their impact - for the first time, deep learning systems are changing how ordinary people work and communicate on a daily basis.

1.4. Why Now?

▹ Session 1: Why Now? - Four Driving Forces

Deep learning is not a new idea - neural networks have existed since the 1950s. What changed is the confluence of four factors that together made the approach viable at scale. None of these alone was sufficient; together they triggered a revolution.

1.4.1. Data

▹ Session 1: Why Now? - Four Driving Forces, Data - Scale Revolution

The internet created the largest dataset in history by accident. Every uploaded image, every written caption, every search query became potential training material. This was crystallised deliberately in benchmark datasets - ImageNet (2009) provided 1.2 million labelled images across 1,000 categories, and GLUE provided a standardised benchmark

for natural language understanding. These benchmarks did not just measure progress, they created it by giving the community a shared target to optimise against.

Crucially, the field also learned that labels are not always necessary. Self-supervised and weakly supervised learning techniques extract signal directly from unlabelled data, massively expanding what counts as usable training material. The web, in its entirety, became a dataset.

Figure 1.3 shows the growth in dataset scale over time. Two aspects deserve attention. First, the left axis is logarithmic - each gridline represents a tenfold increase, so the jump from ImageNet to LAION-5B represents roughly four thousand times more data. Second, the orange line shows that resolution grew alongside quantity - from 28×28 pixels for MNIST to 1024×1024 for LAION-5B, a 1,000-fold increase in pixels per image. The two dimensions of scale - how many images and how much information per image - grew together.

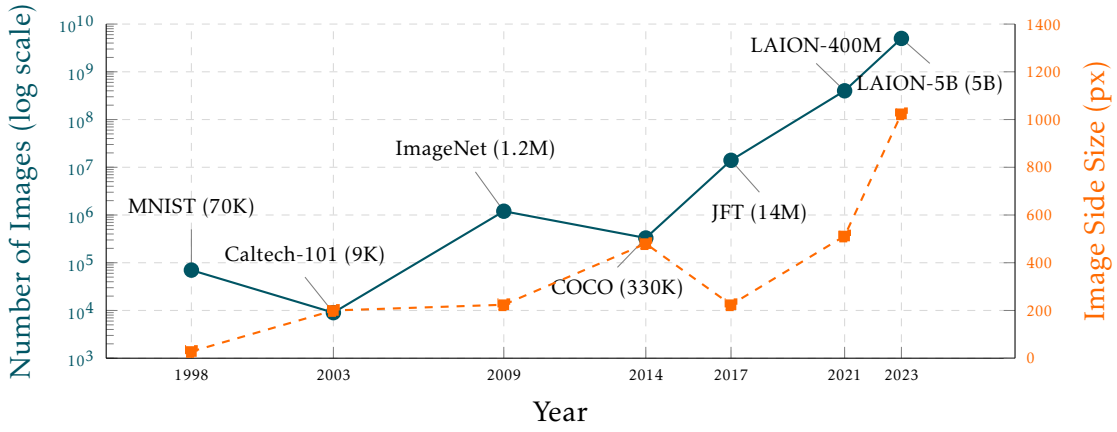

Figure 1.3: Image dataset sizes and resolutions over time. Blue line: number of images (log scale, left axis). Orange line: image side size in pixels (linear scale, right axis).

1.4.2. Compute

▹ Session 1: Compute & Progress

Graphics processing units, designed for the parallel computation of gaming graphics, turned out to be ideally suited for the matrix multiplications at the heart of neural network training. NVIDIA's CUDA programming model made GPUs accessible to researchers without hardware expertise. Custom chips - Google's TPUs, Apple Silicon, and a wave of AI-specific inference hardware - followed, with architectures designed around deep learning from the ground up.

Cloud platforms such as AWS, Microsoft Azure, and Google Cloud democratised access to large-scale compute - a researcher can now rent state-of-the-art hardware by the hour. Learning platforms such as Google Colab, Kaggle Notebooks, and Lightning.ai went further, providing free GPU access directly in the browser with zero setup. As Figure 1.4 shows, GPU performance has grown from single-digit TFLOPS in 2012 to nearly 2,000 TFLOPS in 2024.

1.4.3. Algorithms

▹ Session 1: Why Now? - Four Driving Forces, Compute & Progress

Several targeted innovations removed practical obstacles that had blocked progress for decades. At the architectural level, the multilayer perceptron (MLP), convolutional neu-

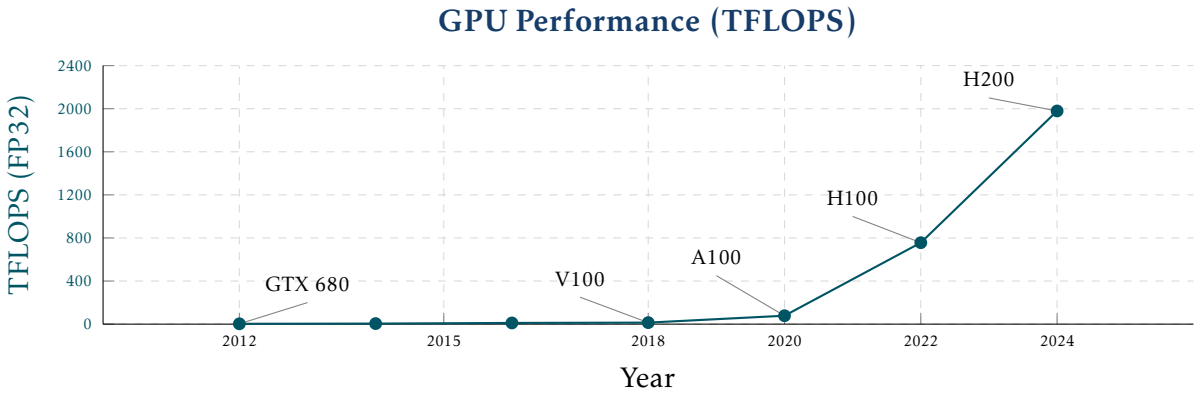

Figure 1.4: GPU performance in TFLOPS (FP32) from 2012 to 2024.

ral network (CNN), and recurrent neural network (RNN) established the foundational building blocks, all trained via the backpropagation algorithm. The ReLU activation function mitigated the vanishing gradient problem that had plagued deep networks. Batch normalisation stabilised training. Dropout provided effective regularisation.

At a higher level, residual connections (ResNets) made it possible to train networks of hundreds of layers. The attention mechanism and the transformer architecture, introduced in 2017, replaced recurrent computation entirely and became the foundation of modern language and vision models.

Figure 1.5 illustrates the impact of these algorithmic advances on the ImageNet benchmark. In 2011, the best classical methods using SVMs achieved around 74% top-5 accuracy. AlexNet's jump to 84.7% in 2012 marked the beginning of the deep learning era. Each subsequent architectural innovation - VGG, ResNet, EfficientNet, ViT - pushed accuracy further, surpassing human performance of approximately 95% around 2015 and reaching over 99% by 2022.

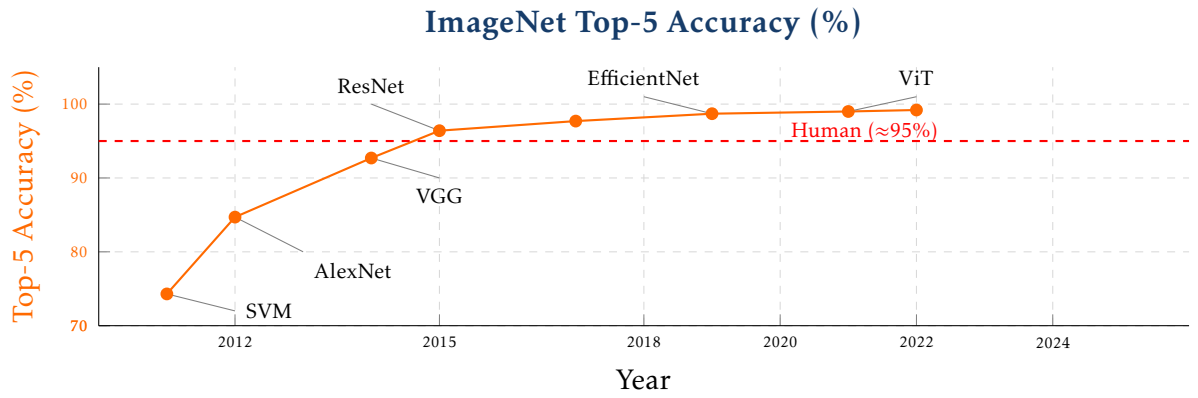

Figure 1.5: ImageNet top-5 accuracy from 2011 to 2022. Each labelled point corresponds to a key architectural innovation. The dashed red line indicates approximate human performance.

1.4.4. Community

Perhaps the most underappreciated factor is the open culture of the deep learning community. Frameworks such as PyTorch, TensorFlow, and JAX are open source,

actively maintained, and freely available. The norm of releasing code alongside papers - through platforms such as Papers with Code, Kaggle, and HuggingFace - means that results are reproducible and immediately buildable upon. Research is shared on arXiv before peer review, and major conferences such as NeurIPS, ICML, and ICLR publish proceedings openly.

This compounding effect - where each breakthrough immediately becomes everyone's foundation - has no parallel in most other scientific fields.

1.5. What Can Deep Learning Do?

▹ Session 1: What Can Deep Learning Do?

Deep learning today touches almost every domain of human activity. Rather than organising applications by the models used, it is more instructive to group them by the type of task they perform.

1.5.1. Language and Communication

Natural language was one of the earliest and remains one of the most impactful application areas. Machine translation systems such as DeepL and Google Translate now produce fluent output across dozens of language pairs, a task that resisted decades of rule-based approaches. Conversational agents - ChatGPT, Claude, Gemini - have brought language models to a mass audience, demonstrating capabilities in reasoning, summarisation, and question answering that were considered out of reach just a few years ago. Code generation tools such as GitHub Copilot and Cursor assist developers by completing, explaining, and generating code in real time.

1.5.2. Vision and Perception

Computer vision was the domain in which deep learning first demonstrated decisive superiority over classical methods, with AlexNet's 2012 ImageNet result marking the turning point. Today deep learning powers image classification, object detection, and semantic segmentation across a vast range of applications. Autonomous driving systems parse complex road scenes in real time, combining camera, lidar, and radar inputs to make driving decisions. Satellite and aerial imagery analysis enables large-scale monitoring of land use, deforestation, and infrastructure.

1.5.3. Healthcare and Medicine

Medical imaging has been transformed by deep learning. Systems trained on large collections of labelled scans can detect tumours, classify X-rays, and segment anatomical structures with accuracy approaching or exceeding that of specialist clinicians. Beyond imaging, deep learning is applied to genomics - identifying disease-associated variants in DNA sequences - and to clinical decision support systems that assist physicians with diagnosis and treatment planning.

1.5.4. Generation

Generative models have emerged as one of the most publicly visible applications of deep learning. Image generation systems such as Stable Diffusion and DALL-E produce photorealistic or stylised images from text descriptions. Video and music

generation - tools such as Suno and Udio - extend this capability to temporal media. The same underlying technology that enables generation also enables detection: deepfake detection systems attempt to identify synthetically generated media, an increasingly important capability as generation quality improves.

1.5.5. Scientific Discovery

Perhaps the most consequential application of deep learning to date is AlphaFold Jumper, Evans, Pritzel, et al. 2021, DeepMind's system for predicting protein three-dimensional structure from amino acid sequence. AlphaFold solved a fifty-year-old open problem in structural biology and earned Demis Hassabis and John Jumper the 2024 Nobel Prize in Chemistry - the first Nobel Prize awarded for work driven by deep learning. Deep learning is also applied to drug discovery - predicting molecular properties and identifying candidate compounds - and to climate and weather modelling, where neural networks are beginning to challenge traditional physics-based simulation approaches.

1.5.6. Decision Making and Control

Reinforcement learning - a branch of deep learning in which agents learn by interacting with an environment - has produced some of the field's most dramatic results. AlphaGo Silver, Huang, Maddison, et al. 2016 defeated the world champion Go player in 2016, a milestone considered a decade away by most experts at the time. Robotic systems learn manipulation and locomotion skills directly from experience, without hand-coded controllers. Recommendation systems - the algorithms that determine what appears in TikTok feeds, Netflix homepages, and Spotify playlists - are among the most economically impactful applications of deep learning, shaping the daily media consumption of billions of people.

2. CNNs in Practice

▹ Session 10

Session 9 introduced the theory and architecture of convolutional neural networks. This session focuses on how CNNs are actually used in practice. The central message is straightforward: you will almost never train a CNN from scratch. Instead, you will download a pretrained model and adapt it to your task. We examine when and how to do this, cover the practical PyTorch tools involved, and then extend beyond image classification to the three major visual tasks that CNNs also power: object detection, semantic segmentation, and instance segmentation.

2.1. Transfer Learning

▹ Session 10: Three Regimes, Why Pretrained Models Work, Feature Extraction vs Fine-tuning, When to Use Which, Practical Tuning

Training a CNN from scratch on every new problem would be enormously wasteful — the low-level visual features learned by early layers are essentially the same regardless of the task. Transfer learning exploits this by reusing weights learned on one task as the starting point for another. This section covers when and how to do this effectively.

2.1.1. Three Regimes

There are three distinct regimes for working with CNNs, each appropriate in different circumstances.

Training from scratch means initialising all weights randomly and optimising the full network on your own dataset. This requires a large labelled dataset (tens of thousands to millions of examples), significant compute (days to weeks on multiple GPUs), and is only justified when you have a genuinely novel architecture, a domain so different that pretrained weights would be useless, or when you are a large organisation with the necessary resources. For most practitioners, training from scratch is not a realistic option.

Pretraining followed by fine-tuning means deliberately designing a two-phase training pipeline. In the first phase, you train on a large **surrogate dataset** — one that is not your final target but is large enough to learn rich visual representations. In the second phase, you fine-tune on your small target dataset. This approach is valuable when no suitable pretrained model exists for your domain, but a related large dataset does. For example, a medical imaging researcher might pretrain on a large general radiology dataset and then fine-tune on a small collection of annotated cases for a specific rare condition. The surrogate dataset does not need to be perfectly aligned with the target task — it just needs to be large enough to learn useful features that transfer.

Downloading and adapting is the dominant modern workflow. Pretrained models are freely available from torchvision, timm, and HuggingFace, trained by large organisations on massive datasets. You download a model, replace its classification head with one appropriate for your task, and either use it as a fixed feature extractor or fine-tune some layers. This approach works surprisingly well even for tasks quite different from the original pretraining, requires relatively little data and compute, and is how the vast majority of real CNN applications are built today.

2.1.2. Why Pretrained Models Work

The reason transfer learning works so well is the hierarchical nature of what CNNs learn. As demonstrated by Zeiler and Fergus 2014, different layers of a CNN learn features at different levels of specificity:

- **Early layers** learn general low-level features: edges, colour gradients, oriented bars. These features are essentially universal — they appear in medical images, satellite imagery, microscopy, and natural photographs alike. A filter that detects a horizontal edge in an ImageNet-trained network is equally useful for detecting edges in a chest X-ray.
- **Later layers** learn increasingly task-specific features: object parts, textures associated with specific categories, combinations of lower-level features that are meaningful for the source task. These features are less transferable — a network trained to classify ImageNet categories will have late-layer features tuned for dogs, cars, and furniture, which may not be useful for classifying skin lesions.

This hierarchical structure means that pretrained weights are almost always a better starting point than random initialisation, even when the target domain differs substantially from the source. The question is not whether to use pretrained weights, but how much of the network to adapt.

Modern pretrained models are trained on increasingly large datasets: ImageNet (1.2M images, 1000 classes), LAION-5B (5.8 billion image-text pairs), and various proprietary

datasets used by large companies. Models trained on larger, more diverse datasets tend to transfer better across a wider range of target tasks.

2.1.3. Feature Extraction vs Fine-tuning

Recall that a CNN trained for classification consists of two functional parts. The **convolutional backbone** — the stack of convolutional and pooling layers — acts as a feature extractor: it maps an input image to a compact feature vector encoding what is present in the image. The **classification head** is typically a single fully connected layer (or a small MLP) that maps this feature vector to class scores. When we adapt a pretrained model, we discard the original head (which outputs scores for the source task's classes) and replace it with a new one sized for our target task.

Once you have a pretrained model, there are two main strategies for adapting it to a new task.

Feature extraction treats the pretrained network as a fixed feature extractor. All pretrained layers are frozen — their weights are not updated during training — and only a new classification head is trained. The pretrained network maps each input image to a feature vector, and the head learns to classify these vectors for the new task. Feature extraction is appropriate when the target dataset is small (hundreds to a few thousand examples), when compute is limited, or when the target domain is similar enough to the source that the pretrained features are directly useful.

Fine-tuning unfreezes some or all of the pretrained layers and trains them alongside the new head, typically with a much smaller learning rate. This allows the network to adapt its pretrained features to the specific characteristics of the target domain, at the cost of needing more data and compute. Fine-tuning is appropriate when the target dataset is large enough to avoid overfitting the deeper layers, or when the target domain is sufficiently different from the source that direct feature transfer is insufficient.

Catastrophic forgetting is a key risk in fine-tuning: if the learning rate is too large, the large noisy gradients from the randomly initialised head propagate back into the backbone and overwrite the pretrained weights, destroying the useful representations built up during pretraining. The standard mitigation is to use a learning rate 10×–100× smaller than the original pretraining learning rate when fine-tuning pretrained layers.

2.1.4. When to Use Which

The choice between feature extraction and fine-tuning depends on two factors: the size of the target dataset and the similarity between the source and target domains.

	Similar domain	**Different domain**
Small dataset	Feature extraction (freeze all, train head)	Feature extraction (try different layer depths)
Large dataset	Fine-tune all layers (small learning rate)	Fine-tune all layers (more epochs, careful tuning)

Feature extraction is the right choice when the target dataset is small. With few examples, fine-tuning the backbone risks overfitting — the network will memorise the training set rather than learning to generalise. If the target domain is similar to the source, the pretrained features transfer directly and a new head trained on top is often

sufficient. If the domain is different, it is worth experimenting with which layers to use as features: earlier layers capture more general low-level structure and may transfer better than later layers which are tuned to source-task semantics.

Fine-tuning becomes appropriate when enough labelled data is available to adapt the backbone without overfitting. With a similar domain, fine-tuning all layers with a small learning rate typically yields the best results. With a different domain, more epochs, a lower learning rate, and more aggressive data augmentation may be needed to fully adapt the representations.

2.1.5. Practical Tuning

The starting point for any transfer learning workflow is always to freeze the entire backbone and train only the new head. Once the head has stabilised — typically after a few epochs — you can begin unfreezing layers progressively from the top, since later layers are the most task-specific and most in need of adaptation. Earlier layers, which encode general low-level features, should remain frozen longer or not be touched at all.

When unfreezing layers, use different learning rates for different parts of the network. The new head, which starts from random initialisation, can tolerate a standard learning rate such as 1e-3. Fine-tuned backbone layers should use a much smaller rate — 1e-5 or lower — to avoid overwriting the pretrained representations. This technique, known as **discriminative learning rates**, is straightforward to implement in PyTorch by passing parameter groups with per-group learning rates to the optimiser.

It also helps to warm up the learning rate for the backbone layers at the start of fine-tuning: begin with a very small value and increase it gradually over the first few epochs. This gives the randomly initialised head time to stabilise and produce reasonable gradients before the backbone weights are significantly updated.

Early stopping is essential when fine-tuning — the combination of a small dataset and a powerful pretrained model means overfitting can happen within just a few epochs. Monitor validation loss carefully and save the checkpoint at the best validation performance rather than the final one.

Finally, data augmentation deserves particular attention when the target dataset is small. Standard augmentations — random crops, horizontal flips, colour jitter — are always worth applying. For domains far from natural images, domain-specific augmentations can be added on top and often provide a significant boost.

2.2. Practical CNN Training

This section covers the practical PyTorch tools for putting transfer learning into practice: where to find pretrained models, how to adapt them in code, and how to preprocess images correctly.

▹ Session 10: torchvision and timm, Adapting Pretrained Model, Image Transforms Pipeline

2.2.1. Sources of Pretrained Models

Before writing any code, two decisions need to be made: which model architecture to use, and which pretrained weights to load. These choices interact and depend on your task, your dataset, and your available compute.

Choosing an architecture. The right architecture depends on several factors. Accuracy is the obvious one, but it should not be the only consideration. Larger models

— more layers, more parameters — generally achieve higher accuracy on standard benchmarks, but they are slower to train and run, consume more memory, and have a larger carbon footprint. For transfer learning on a small dataset, a smaller model is often the better choice: it is faster to fine-tune, less likely to overfit, and easier to deploy. A good starting point is to identify the accuracy-efficiency tradeoff that fits your constraints, then pick an architecture in that range. Resources such as the Papers with Code maintain up-to-date leaderboards comparing architectures on standard benchmarks, with links to the original papers and open-source implementations. This makes it straightforward to survey the accuracy-efficiency landscape, identify architectures worth considering, and find the corresponding code and pretrained weights.

Choosing weights. Once an architecture is chosen, the weights determine what features the backbone has learned. The common assumption — that weights performing well on the original task will transfer well to a new one — does not always hold. Transfer performance depends on how similar the source and target domains are, how the model was trained (the training recipe matters as much as the dataset), and sometimes on factors that are hard to predict in advance. Weights trained with modern recipes using mixup, label smoothing, or self-supervised objectives sometimes transfer better than older ImageNet checkpoints even for the same architecture. The safest approach is to treat weight selection as a hyperparameter: if compute allows, benchmark a few candidate weights on a small validation set rather than assuming the highest ImageNet accuracy corresponds to the best transfer performance.

Sources of pretrained models. Three main sources are available for PyTorch users.

torchvision.models is the official PyTorch model zoo, maintained by the PyTorch team. It covers a curated set of standard architectures — ResNet, VGG, EfficientNet, Vision Transformers — pretrained on ImageNet with well-documented training configurations. Because it is officially maintained, it is the most reliable source in terms of correctness and long-term stability. The API is simple and consistent, and each model entry documents exactly which weights are available and what normalisation statistics they require. The main limitation is breadth: torchvision covers the established classics but does not track the research frontier closely.

```
from torchvision.models import resnet50, ResNet50_Weights
model = resnet50(weights=ResNet50_Weights.DEFAULT)
```

timm (PyTorch Image Models) is a community-maintained library with over 700 pretrained models, including many architectures and weight variants not available in torchvision. It is the standard resource in vision research when you need something beyond the classics. A key advantage is that timm standardises the interface across architectures, so switching between models requires minimal code changes. The trade-off is that being community-maintained means quality and documentation can vary across models. In practice, the most popular models in timm are well-tested and widely used. Weights in timm often come from multiple sources and training recipes, which makes the choice richer but also more demanding — the documentation and model cards should be consulted carefully.

```
import timm
model = timm.create_model('resnet50', pretrained=True)
```

HuggingFace is a large model hub hosting contributions from both the research community and industry. For pure CNN-based vision tasks it largely overlaps with timm, but it becomes essential when working with more recent model families that combine vision with language — such as CLIP or DINO — based on architectures called Vision Transformers which go beyond what our introductory course covers. For now it is worth knowing that HuggingFace is where you would find them, and that the transformers library provides a consistent interface across a very wide range of model types. The quality and reliability of models on HuggingFace varies significantly — always check the model card and prefer models from established organisations or with a strong download and community track record.

For straightforward CNN transfer learning tasks, torchvision is the natural first stop for its reliability and simplicity. timm is the next step when you need more architectural choices or more recent weights. HuggingFace becomes relevant when your task requires vision-language models or when you are looking for community-contributed specialised weights.

2.2.2. Adapting a Pretrained Model

Once a pretrained model is loaded, the standard adaptation workflow has two steps: replace the classification head with one sized for your target task, and decide which parameters to train. In PyTorch this is done by manipulating `requires_grad` flags on parameter groups and constructing the optimiser accordingly. The following example shows feature extraction with a ResNet-50 backbone.

```
model = resnet50(weights=ResNet50_Weights.DEFAULT)

# freeze all pretrained layers
for param in model.parameters():
    param.requires_grad = False

# replace FC head with new classifier
num_features = model.fc.in_features
model.fc = nn.Linear(num_features, num_classes)

# train only the new head
optimizer = torch.optim.Adam(model.fc.parameters(), lr=1e-3)
```

In ResNet, the classification head is a single fully connected layer accessible as `model.fc` — it maps the 2048-dimensional output of the backbone to class scores. Replacing it with a new `nn.Linear` sized for your number of classes, while freezing all other parameters, gives a feature extraction setup where only the new head is trained.

For fine-tuning, selected backbone layers can be unfrozen and trained alongside the head, typically with a much smaller learning rate. The following example unfreezes the last ResNet block (`layer4`) and uses discriminative learning rates — a higher rate for the new head and a lower one for the unfrozen backbone layers.

```
# unfreeze last ResNet block
for param in model.layer4.parameters():
    param.requires_grad = True
```

```
# different learning rates per layer group
optimizer = torch.optim.Adam([
    {'params': model.layer4.parameters(), 'lr': 1e-5},
    {'params': model.fc.parameters(),     'lr': 1e-3},
])
```

All the basics from earlier sessions still apply: `model.train()` and `model.eval()` for mode switching, `optimizer.zero_grad()` before each backward pass, and the standard training loop.

2.2.3. Image Transforms Pipeline

Before images can be fed to a CNN, they must be converted into the format the network expects. This involves a pipeline of transformations applied to each image, constructed using `torchvision.transforms.Compose`.

Three transformations are always required. `ToTensor` converts a PIL image or NumPy array to a PyTorch tensor and scales pixel values from $[0, 255]$ to $[0, 1]$. `Resize` brings the image to the spatial dimensions the network expects — most ImageNet-pretrained models require 224×224 input, and feeding a different size will either cause an error or silently produce wrong results. `Normalize` then standardises each channel by subtracting the mean and dividing by the standard deviation.

Normalisation is important because the pretrained model was trained on data preprocessed with specific statistics — using different values shifts the input distribution away from what the network expects, degrading performance. When using a pretrained model, the normalisation statistics must therefore match those used during pretraining, not be computed from your own dataset. Both torchvision and timm document the correct statistics for each pretrained model; for ImageNet-pretrained models the standard values are mean `[0.485, 0.456, 0.406]` and standard deviation `[0.229, 0.224, 0.225]` per channel. Models pretrained on specialised data — medical images, satellite imagery, microscopy — can have statistics that differ substantially from these values, and using ImageNet statistics with such models will hurt performance.

Data augmentation applies random perturbations to training images to artificially increase the effective size of the dataset and improve generalisation. `RandomResizedCrop` crops a random region of the image and resizes it to the target size, encouraging the network to be robust to scale and position. `RandomHorizontalFlip` mirrors the image with 50% probability, useful for tasks where left-right symmetry holds. `ColorJitter` randomly varies brightness, contrast, saturation, and hue, making the model robust to lighting conditions. These are among the most commonly used augmentations, but many more are available — including `RandomRotation`, `RandomGrayscale`, `GaussianBlur`, and `RandomErasing` — and the full list is documented in the torchvision transforms documentation.

The right choice of augmentations is task-dependent and is one of the places in the deep learning pipeline where domain expertise and careful thinking about the data genuinely matter. Horizontal flipping is appropriate for natural images but not for tasks where orientation carries meaning — reading text, analysing ECGs, or classifying asymmetric skin lesions. Aggressive colour jitter helps when lighting conditions vary but can destroy diagnostically relevant colour information in histology or dermatology images. Geometric distortions such as random rotation are useful for aerial or satellite

imagery where objects can appear at any angle, but inappropriate for tasks where vertical orientation is meaningful. There is no universal recipe: the best augmentation strategy comes from understanding what variability is present in the real deployment environment and what properties of the image are irrelevant to the label. A useful heuristic is to ask for each augmentation — could this transform turn a correctly labelled image into a wrongly labelled one? If yes, it should not be used.

Regardless of which augmentations are chosen, they must only be applied during training. At validation and test time the transform pipeline should contain only deterministic operations — resizing, cropping, and normalisation. Random augmentations at validation time would make the loss noisy across epochs, undermining early stopping, and would make test results non-reproducible.

```
from torchvision import transforms

train_transform = transforms.Compose([
    transforms.RandomResizedCrop(224),
    transforms.RandomHorizontalFlip(),
    transforms.ColorJitter(brightness=0.2, contrast=0.2),
    transforms.ToTensor(),
    transforms.Normalize(mean=[0.485, 0.456, 0.406],
                         std=[0.229, 0.224, 0.225])
])

val_transform = transforms.Compose([
    transforms.Resize(256),
    transforms.CenterCrop(224),
    transforms.ToTensor(),
    transforms.Normalize(...)
])
```

The transform pipeline is applied on the fly during data loading — each image is transformed as it is fetched by the `DataLoader`, not pre-processed and stored on disk. This has an important consequence: every time an image is loaded, a fresh set of random transformations is applied. The same image will be cropped to a different region, flipped or not, with different brightness, each time it appears in training. Over many epochs the network effectively sees many slightly different versions of each image, which is precisely what gives augmentation its regularising effect — it makes it harder for the network to memorise individual training examples and forces it to learn features that are robust to the kinds of variation the augmentations introduce.

The order of transformations in the pipeline matters. Geometric transformations such as `RandomResizedCrop` and `RandomHorizontalFlip` should come first, while the image is still in PIL format. `ToTensor` should follow, converting the image to a tensor, after which pixel-level operations such as `Normalize` and `ColorJitter` are applied. Applying `Normalize` before `ToTensor` would fail outright since `Normalize` operates on tensors. More subtly, applying colour augmentations before resizing is slightly wasteful — you are jittering pixels that may be discarded by the crop. Following the order shown in the code example above is a safe and standard convention.

2.3. Beyond Classification

▹ Session 10: Vision Tasks, Object Detection, Evaluation – IoU and mAP, Semantic Segmentation, Instance Segmentation, Annotation Cost

Image classification assigns one or more labels to an entire image. Many real-world applications require richer outputs: locating objects, labelling every pixel, or distinguishing individual instances of the same class. Figure 2.1 illustrates the four main visual tasks. This section provides a brief introduction to each — enough to understand what these tasks involve and how CNNs are applied to them. A full treatment, including the details of modern architectures and training procedures, is beyond the scope of these notes and is typically covered by a dedicated course on computer vision.

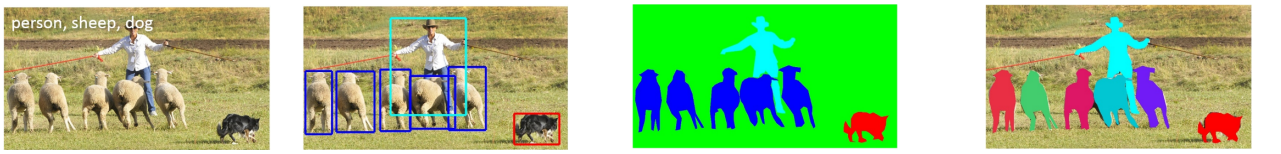

Figure 2.1: Four main visual tasks. Classification assigns one ro more labels to the whole image. Object detection localises and classifies multiple objects with bounding boxes. Semantic segmentation assigns a class label to every pixel. Instance segmentation additionally distinguishes individual objects of the same class.

All four tasks share a common pattern: a pretrained CNN backbone extracts features, and a task-specific head produces the required output. This is why the transfer learning principles from Section 1 apply directly — the backbone can be pretrained on ImageNet and fine-tuned for any of these tasks.

2.3.1. Object Detection

Object detection requires the network to output a variable number of predictions, one per object present in the image. Each prediction consists of a bounding box — defined by four coordinates, typically the centre position, width, and height — a class label identifying what type of object it is, and a confidence score reflecting how certain the network is about that prediction. Unlike classification, the number of objects is not known in advance and can vary from image to image. A street scene might contain dozens of pedestrians, cars, and traffic signs simultaneously, each needing to be localised and identified independently.

Two-stage detectors. The R-CNN family of detectors (Ren et al. 2015) uses a two-stage approach. In the first stage, a **Region Proposal Network (RPN)** scans the feature map and proposes candidate regions that might contain objects, using anchor boxes of different scales and aspect ratios at each spatial location. In the second stage, each proposed region is extracted, pooled to a fixed size (RoI pooling), and passed through a classifier and bounding box regressor. Faster R-CNN introduced the RPN, making the whole pipeline end-to-end trainable and reducing inference time to around 0.2 seconds per image. Two-stage detectors generally achieve higher accuracy but are slower than single-stage approaches.

Single-stage detectors. The YOLO (Redmon et al. 2016) (You Only Look Once) family of models predict bounding boxes and class probabilities directly from the feature map

in a single forward pass. YOLO divides the image into an $S \times S$ grid; each cell predicts B bounding boxes with confidence scores and C class probabilities, producing an $S \times S \times (B \times 5 + C)$ output tensor. This single-pass design makes YOLO extremely fast and suitable for real-time applications. Modern YOLO versions (now at v10/v11) have largely closed the accuracy gap with two-stage detectors while maintaining high speed.

Evaluation. Object detection is evaluated using **Intersection over Union (IoU)** and **mean Average Precision (mAP)**.

IoU measures the overlap between a predicted bounding box and the ground truth box (Figure **??**):

$$\text{IoU} = \frac{|\text{intersection}|}{|\text{union}|} \in [0, 1].$$

A prediction is counted as correct if its IoU with the ground truth exceeds a threshold, typically 0.5.

Average Precision (AP) for a single class summarises the precision-recall tradeoff across all possible confidence thresholds. As the threshold is lowered, more predictions are accepted: recall increases because more true objects are found, but precision falls because more false positives are included. Sweeping the threshold from high to low traces out the precision-recall curve. In practice, AP is not computed as the area under the smooth curve but as a stepped approximation: at each discrete recall level corresponding to an actual detection, the precision value is recorded, producing a staircase. AP is then the area under this staircase, equivalent to a weighted sum of precision values at each step. Mean AP (mAP) averages AP across all classes, giving a single number summarising detector performance across the full recognition vocabulary.

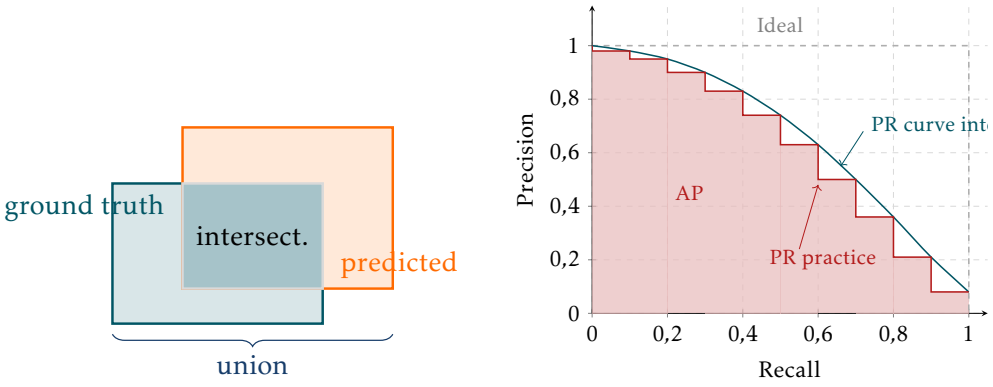

Figure 2.2: Left: IoU measures the overlap between predicted and ground truth bounding boxes. A prediction is counted as correct if IoU exceeds a threshold, typically 0.5. Right: Precision-recall curve for a single class. The ideal detector (dashed) maintains perfect precision at all recall levels. A real detector traces a smooth curve that falls as the confidence threshold is lowered. Average Precision (AP) is approximated as the area under the stepped interpolation; mean AP (mAP) averages this across all classes.

2.3.2. Semantic Segmentation

Semantic segmentation assigns a class label to every pixel in the image, producing an output label map of the same spatial size as the input. All pixels belonging to the

same class receive the same label, regardless of whether they belong to different object instances.

The dominant architecture for semantic segmentation is the **encoder-decoder**. The encoder is a CNN backbone that progressively reduces spatial resolution while extracting rich features. The decoder upsamples the low-resolution feature maps back to the input resolution, recovering spatial detail. The challenge is that the aggressive downsampling in the encoder discards fine spatial information that is needed to produce accurate pixel-level predictions.

U-Net (Ronneberger, Fischer, and Brox 2015), illustrated in Figure 2.3, addressed this by introducing **skip connections** between corresponding encoder and decoder layers. These connections concatenate high-resolution encoder features directly with the upsampled decoder features at each resolution level, allowing the decoder to recover spatial detail that would otherwise be lost. Originally developed for medical image segmentation, U-Net remains one of the most widely used segmentation architectures.

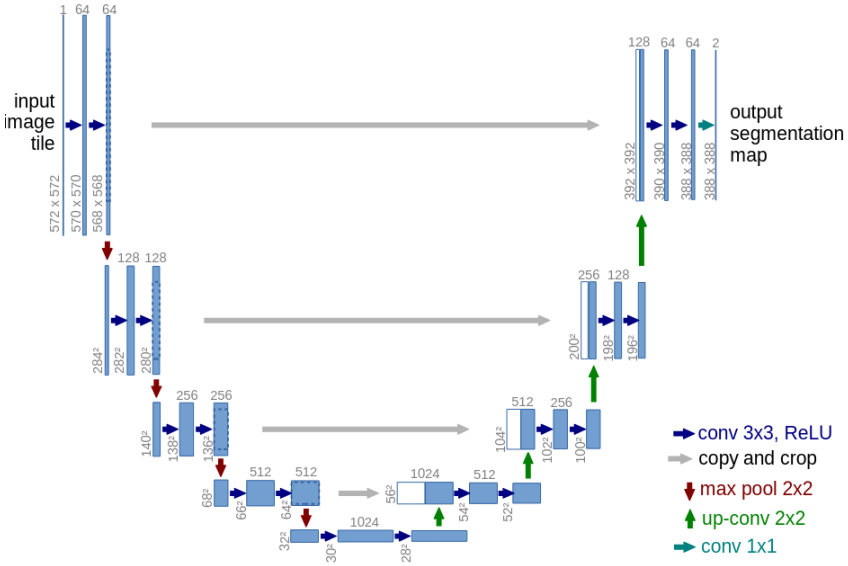

Figure 2.3: The U-Net architecture. The encoder (left) progressively halves spatial resolution while doubling feature channels. The decoder (right) upsamples back to the input resolution. Skip connections (horizontal arrows) concatenate encoder and decoder features at each resolution level, recovering fine spatial detail lost during encoding.

Semantic segmentation is evaluated using **mean Intersection over Union (mIoU)**: IoU is computed per class (comparing predicted and ground truth pixel masks for each class) and averaged across all classes.

2.3.3. Instance Segmentation

Instance segmentation goes beyond semantic segmentation by distinguishing individual object instances. Two people standing next to each other receive different instance masks even though they belong to the same class. The output is a set of per-instance binary masks, each paired with a class label and confidence score.

Panoptic segmentation unifies semantic and instance segmentation into a single output: background regions such as sky, road, and grass receive semantic labels, while foreground objects receive individual instance masks. It has become the modern standard for comprehensive scene understanding.

For practitioners, several powerful tools are freely available. **SAM** (Segment Anything Model) (Kirillov et al. 2023) is a foundation model trained on one billion masks that can segment any object in any image given a simple prompt — a point, a bounding box, or a text description — without task-specific fine-tuning. SAM has become widely used both for inference and as an annotation assistance tool, dramatically reducing the cost of creating segmentation datasets. It is available via the Meta AI GitHub repository. **Detectron2** (**Wu2019**), also from Meta AI, is a full object detection and segmentation library implementing Mask R-CNN and many other architectures, available at GitHub.

2.3.4. Annotation Cost

A practical consideration often underestimated by practitioners is annotation cost. The annotation requirements differ dramatically across tasks, as shown in Table 1.

Task	Annotation type	Time per image	Example dataset
Classification	Image-level label	Seconds	ImageNet
Detection	Bounding boxes	Minutes	COCO
Semantic segmentation	Pixel-wise labels	30–90 min	Cityscapes
Instance segmentation	Per-instance masks	Hours	COCO

Table 1: Annotation cost comparison across visual tasks.

In practice, data is almost always the bottleneck — not the model. Given a well-annotated dataset, training a competitive model is straightforward: the architectures are well understood, pretrained weights are freely available, and the training procedure is largely standardised. The hard part is getting the data right. A poorly annotated dataset will produce a poorly performing model regardless of how sophisticated the architecture is, and no amount of hyperparameter tuning will fix labels that are inconsistent, incomplete, or simply wrong.

This means that investing time and effort in data collection, cleaning, and annotation pays far greater dividends than optimising the model. Before reaching for a more complex architecture, it is worth asking whether the dataset is the real limiting factor. More data, better labels, and more careful quality control will often outperform a fancier model trained on the same flawed dataset.

Several practical strategies help manage annotation cost. **SAM** can generate high-quality segmentation masks interactively from simple point or bounding box prompts, reducing mask annotation time by an order of magnitude. **Semi-supervised and weakly supervised methods** exploit a small amount of expensive annotations combined with a large amount of cheap ones — for example using image-level labels to supervise a detection model. **Synthetic data** generated from 3D simulations or procedural pipelines provides automatically annotated training examples, particularly useful in domains where real annotations are prohibitively expensive such as autonomous driving or robotics. The common thread is to be clever about data: think carefully about what annotations are truly needed, use the cheapest label type that is sufficient for the task, and leverage tools that reduce the human effort required.

Acknowledgements

These lecture notes were prepared with the assistance of Claude, a large language model developed by Anthropic. All content has been reviewed, edited, and approved by the author.

References

Bengio, Yoshua, Olivier Delalleau, and Nicolas Le Roux (2004). "The Curse of Highly Variable Functions for Local Kernel Machines". In: *Advances in Neural Information Processing Systems*.

Deng, Jia et al. (2009). "ImageNet: A Large-Scale Hierarchical Image Database". In: *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*.

Fukushima, Kunihiko (1980). "Neocognitron: A Self-Organizing Neural Network Model for a Mechanism of Pattern Recognition Unaffected by Shift in Position". In: *Biological Cybernetics* 36, pp. 193–202.

Goodfellow, Ian, Yoshua Bengio, and Aaron Courville (2016). *Deep Learning*. Available at https://www.deeplearningbook.org. MIT Press.

Hinton, Geoffrey E., Simon Osindero, and Yee-Whye Teh (2006). "A Fast Learning Algorithm for Deep Belief Nets". In: *Neural Computation* 18.7, pp. 1527–1554.

Hochreiter, Sepp and Jürgen Schmidhuber (1997). "Long Short-Term Memory". In: *Neural Computation* 9.8, pp. 1735–1780.

Hopfield, John J. (1982). "Neural Networks and Physical Systems with Emergent Collective Computational Abilities". In: *Proceedings of the National Academy of Sciences* 79.8, pp. 2554–2558.

Jumper, John, Richard Evans, Alexander Pritzel, et al. (2021). "Highly Accurate Protein Structure Prediction with AlphaFold". In: *Nature* 596, pp. 583–589.

Kirillov, Alexander et al. (2023). "Segment Anything". In: *Proceedings of the IEEE International Conference on Computer Vision*, pp. 4015–4026.

Krizhevsky, Alex, Ilya Sutskever, and Geoffrey E. Hinton (2012). "ImageNet Classification with Deep Convolutional Neural Networks". In: *Advances in Neural Information Processing Systems*. Vol. 25.

LeCun, Yann, Yoshua Bengio, and Geoffrey Hinton (2015). "Deep Learning". In: *Nature* 521, pp. 436–444.

LeCun, Yann, Léon Bottou, et al. (1998). "Gradient-Based Learning Applied to Document Recognition". In: *Proceedings of the IEEE* 86.11, pp. 2278–2324.

Lighthill, James (1973). *Artificial Intelligence: A General Survey*. Tech. rep. Available at https://www.aiai.ed.ac.uk/events/lighthill1973/lighthill.pdf. Science Research Council.

Ministry of International Trade and Industry (1982). *Outline of Research and Development Plans for Fifth Generation Computer Systems*. Tech. rep. Institute for New Generation Computer Technology.

Minsky, Marvin and Seymour Papert (1969). *Perceptrons: An Introduction to Computational Geometry*. MIT Press.

Nair, Vinod and Geoffrey E. Hinton (2010). "Rectified Linear Units Improve Restricted Boltzmann Machines". In: *Proceedings of the International Conference on Machine Learning*.

Raina, Rajat, Anand Madhavan, and Andrew Y. Ng (2009). "Large-Scale Deep Unsupervised Learning Using Graphics Processors". In: *Proceedings of the International Conference on Machine Learning*.

Redmon, Joseph et al. (2016). "You Only Look Once: Unified, Real-Time Object Detection". In: *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, pp. 779–788.

Ren, Shaoqing et al. (2015). "Faster R-CNN: Towards Real-Time Object Detection with Region Proposal Networks". In: *Advances in Neural Information Processing Systems*. Vol. 28.

Ronneberger, Olaf, Philipp Fischer, and Thomas Brox (2015). "U-Net: Convolutional Networks for Biomedical Image Segmentation". In: *Medical Image Computing and Computer-Assisted Intervention*, pp. 234–241.

Rosenblatt, Frank (1958). "The Perceptron: A Probabilistic Model for Information Storage and Organization in the Brain". In: *Psychological Review* 65.6, pp. 386–408.

Rumelhart, David E., Geoffrey E. Hinton, and Ronald J. Williams (1986). "Learning Representations by Back-propagating Errors". In: *Nature* 323, pp. 533–536.

Silver, David, Aja Huang, Chris J. Maddison, et al. (2016). "Mastering the Game of Go with Deep Neural Networks and Tree Search". In: *Nature* 529, pp. 484–489.

Vapnik, Vladimir (1995). *The Nature of Statistical Learning Theory*. Springer.

Vaswani, Ashish et al. (2017). "Attention Is All You Need". In: *Advances in Neural Information Processing Systems*. Vol. 30.

Werbos, Paul (1974). "Beyond Regression: New Tools for Prediction and Analysis in the Behavioral Sciences". PhD thesis. Harvard University.

Widrow, Bernard and Marcian E. Hoff (1960). "Adaptive Switching Circuits". In: *IRE WESCON Convention Record*. Vol. 4, pp. 96–104.

Zeiler, Matthew D. and Rob Fergus (2014). "Visualizing and Understanding Convolutional Networks". In: *European Conference on Computer Vision*, pp. 818–833.